

42112 Mathematical Programming Modelling — Re-exam Study Guide v2

Exam date: Tuesday, June 2, 2026, room 175, building 358 **Guide updated:** May 23, 2026 — reflects the revised exam announcement

What changed from the earlier announcement

	First announcement	Final announcement
Prep time	20 min, no notes	30 min, notes allowed in prep room
Exam length	Unspecified	25 minutes
Part 1	Questions on Julia/JuMP submission	Questions on SIP production planning submission
Part 2	Random from 5 assignments	Random from 5 assignments (no notes when presenting)
Assignment pool	Santa, TA, Tennis, Food Festival, Wedding	Micro Brewery 2, TA, Tariff Rates, Tennis, Wedding

Key implication of notes in prep room: You do not need to memorise formulas. But you must understand each model well enough to re-explain it at the whiteboard without reading. Internalise the logic; the notation you can reconstruct.

PRIORITY: SIP Production Planning pre-oral submission

The examiner spends Part 1 of your 25 minutes on this. You have not yet shared the problem or data file with me. Upload them and I will: - Help formulate the full mathematical model - Write and debug the Julia/JuMP code - Identify the likely follow-up questions

Treat the SIP submission as your highest priority task right now.

How to use this guide

For each of the 5 assignments you need to be able to: 1. Name the **classical OR problem** it is a variant of 2. Write the **full model** from scratch (Sets $\rightarrow$ Parameters $\rightarrow$ Variables $\rightarrow$ Objective $\rightarrow$ Constraints) 3. Explain each constraint in one plain-English sentence 4. Answer extensions and “what if” modifications

Sections 1–5 give foundations and toolkit. Read once. Then drill section 6 on paper without looking until it feels automatic.

1. Exam structure and strategy

What happens on the day

1. Arrive at your allotted time, room 175. Draw one of 5 assignments at random.
2. Go to **prep room for 30 minutes — notes allowed**.
3. Return for the **25-minute oral**, two parts:
 - **Part 1:** Examiner asks questions about your **SIP pre-oral submission** (code correctness, constraint explanations, extensions).
 - **Part 2:** Present the drawn assignment at the **whiteboard without reading notes**. Examiner asks follow-up questions.

What scores points

- Starting Part 2 with the five headings on the board *before* any math
- Naming the classical OR problem type early
- Explaining each constraint in plain English as you write it
- Defending your SIP code line by line in Part 1
- Reasoning through “I’ve forgotten the exact detail, but I can derive it from...”

What loses points

- Writing Julia code on the whiteboard instead of math
 - Mixing up indices in summations
 - Bluffing when you don’t know
-

2. The standard model template

Memorise the five headings. Write them on the board first every time.

$Setse.g.b \in BeerTypes, m \in Months = 1, \dots, 12$
 $Parameterse.g.Demand_{b,m}, StorageCost = 0.1$
 $Variablese.g.x$

Always say what each variable means in words when you introduce it.

3. LP and MIP — the minimum you need

LP: continuous variables, linear objective and constraints, solves in polynomial time.

MIP: at least one variable is integer or binary, NP-hard in general. Solved by Branch & Bound: solve LP relaxations, branch on fractional variables, prune subproblems that can't improve the best known integer solution.

LP relaxation: drop integrality requirements. Gives an upper bound (max) or lower bound (min). The gap between this and the best integer solution tells you how hard the instance is.

4. Julia/JuMP cheat sheet

```
using JuMP, HiGHS
m = Model(HiGHS.Optimizer); set_silent(m)

@variable(m, x[1:N] >= 0)           # continuous
@variable(m, y[1:N], Bin)          # binary
@variable(m, z[1:N] >= 0, Int)      # integer

@objective(m, Min, sum(c[i]*x[i] for i=1:N))

# Constraint for all i
@constraint(m, [i=1:N], sum(A[i,j]*x[j] for j=1:N) <= b[i])

# Conditional constraint (only generate when condition holds)
@constraint(m, [i=1:N, j=1:N; i<j && Conflict[i,j]==1], x[i]+x[j] <= 1)

# Inline if - used for first period / cyclic boundary conditions
(m > 1 ? s[b,m-1] : InitStorage[b]) # Micro Brewery 2: initial storage
(s > 1 ? n[g,s-1] : n[g,S])         # Tariff Rates: cyclic (slot 0 = slot S)

optimize!(m)
println(termination_status(m), " Obj: ", objective_value(m))
```

5. The modeling tricks toolkit

Trick 1 — Binary indicator

$y \in \{0,1\}$ represents any yes/no decision.

Trick 2 — Fixed charge / activation link

Activity x only possible when $y = 1$: $x \leq B \cdot y$, $y \in \{0,1\}$ B is an upper bound on x .

Trick 3 — At most one from a group

$\sum_i y_i \leq 1$ Used in Micro Brewery 2: at most one beer type brewed per month.

Trick 4 — Inventory balance constraint

$s_{\{b,m\}} = s_{\{b,m-1\}} + x_{\{b,m\}} - \text{Demand}_{\{b,m\}}$ $b, m > 1$ First period uses the given initial stock instead of $s_{\{b,0\}}$.

Trick 5 — Cyclic boundary

When a schedule repeats (period 0 = period H): $nRun_{\{p,0\}} := nRun_{\{p,H\}}$ in the constraint expression. Used in Tariff Rates.

Trick 6 — Startup counting

Count how many generators turn ON per period (not total running): $nStart_{\{p,h\}} = nRun_{\{p,h\}} - nRun_{\{p,h-1\}}$, $nStart_{\{p,h\}} \geq 0$ The 0 ensures shutdowns are ignored ($nStart$ only counts increases in $nRun$).

Trick 7 — Consecutive blocks (TA Workplan)

$y_{\{ta,p,d\}} = 1$ if TA ta starts in slot p on day d . - $\sum_p y_{\{ta,p,d\}} = 1$ ta, d
- $x_{\{ta,p,d\}} = x_{\{ta,p-1,d\}} + y_{\{ta,p,d\}} - \sum_p x_{\{ta,p,d\}} = 2 \cdot \sum_p y_{\{ta,p,d\}}$

Trick 8 — Flow conservation (Tennis)

$\sum_{p2} x_{\{p2,p\}} + s_p = \sum_{p2} x_{\{p,p2\}} + e_p$ p

Trick 9 — Soft constraints via slack (Wedding Planner)

Replace $LHS \leq RHS$ with $LHS \leq RHS + \text{slack}$, $\text{slack} \geq 0$, penalised in objective.

Trick 10 — Linearise binary product (Wedding Planner)

Replace $x \cdot x$ (nonlinear) with variable y and: $y \leq x_{\{g1,t\}}$; $y \leq x_{\{g2,t\}}$

6. The five assignments

§6.5 Micro Brewery 2 — Assignment 5.1

Classical OR problem: Production planning / Lot sizing (MILP). The “lot sizing” name comes from the key constraint: you can only produce one product per period. This turns a pure LP (original Micro Brewery §4.2) into a MIP.

Story (one sentence)

Plan a year of beer production across 3 types and 12 months to meet monthly demand while minimising storage costs, given that you can only brew one type of beer per month and have limited storage capacity.

Data

Demand (litres):

Month	TSP-Stout	Knapsack-Dark	Set-Partitioning-Light
Jan	35	15	5
Feb	20	10	20
Mar	15	20	20
Apr	45	15	35
May	25	15	35
Jun	65	55	80
Jul	40	90	60
Aug	50	80	30
Sep	35	25	35
Oct	85	45	20
Nov	50	5	20
Dec	55	30	40

Other parameters: - Brewing capacity: 120 litres/month (any single type)
- Storage capacity: 300 litres total across all types, per month - Storage cost: €0.10 per litre per month (all types) - Initial stock: TSP-Stout = 25L, Knapsack-Dark = 65L, Set-Partitioning-Light = 75L

Sets

- **b BeerTypes** = {1=TSP-Stout, 2=Knapsack-Dark, 3=Set-Partitioning-Light}
- **m Months** = {1, ..., 12}

Parameters

- **Demand_{b,m}**: litres of beer b demanded in month m (from table)
- **Capacity** = 120 (max litres brewed per month)
- **StorageCap** = 300 (max total litres stored per month)
- **StorageCost** = 0.1 (€/litre/month)
- **InitStock_b**: initial stock in litres (25, 65, 75 for the three types)

Decision variables

(Using the exact variable names from the official course answer key)

- **x_{b,m}** 0: litres of beer b produced in month m
- **y_{b,m}** 0: litres of beer b **in storage at the end** of month m
- **q_{b,m}** {0,1}: 1 if beer type b is brewed in month m

[!] **Variable naming alert:** The teacher uses y for storage and q for the binary selector — the opposite of what you might expect. Know this cold before the whiteboard.

Objective

Minimise total storage cost:

$$\text{minCost} \cdot \sum_b \sum_m y_{b,m}$$

Constraints

- (i) **Inventory balance** — storage at end of month = previous storage + production – demand:

$$y_{b,m} = (m > 1 ? y_{b,m-1} : \text{InitialStorage}_b) + x_{b,m} - \text{Demand}_{b,m} \forall b, m$$

- (ii) **Production only if type selected** (fixed charge link — also enforces capacity):

$$x_{b,m} \leq \text{BrewCap} \cdot q_{b,m} \forall b, m$$

(This single constraint does two jobs: it links x to q AND caps production at BrewCap . No separate capacity constraint needed.)

- (iii) **Total storage capacity:**

$$\sum_b y_{b,m} \leq \text{StoreCap} \forall m$$

- (iv) **At most one beer type per month:**

$$\sum_b q_{b,m} \leq 1 \forall m$$

Julia/JuMP code (matches teacher's official solution exactly)

```
using JuMP, HiGHS

Months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
Beers = ["TSP-Stout", "Knapsack-Dark", "SetPartioning-Light"]
M = length(Months); B = length(Beers)

BrewCap = 120; StoreCap = 300; Cost = 0.1
Demand = [35 20 15 45 25 65 40 50 35 85 50 55;
          15 10 20 15 15 55 90 80 25 45 5 30;
          5 20 20 35 35 80 60 30 35 20 20 40]
InitialStorage = [25 65 75]

microbreweri2 = Model(HiGHS.Optimizer)

@variable(microbreweri2, 0 <= x[1:B, 1:M])           # production
@variable(microbreweri2, 0 <= y[1:B, 1:M])           # storage at end of month
@variable(microbreweri2, q[1:B, 1:M], Bin)           # which beer brewed this month

@objective(microbreweri2, Min, sum(Cost*y[b,m] for b=1:B, m=1:M))

# Inventory balance
@constraint(microbreweri2, [b=1:B, m=1:M],
    y[b,m] == (m > 1 ? y[b,m-1] : InitialStorage[b]) + x[b,m] - Demand[b,m])

# Production link + capacity (one constraint does both)
@constraint(microbreweri2, [b=1:B, m=1:M], x[b,m] <= BrewCap * q[b,m])

# Total storage capacity
@constraint(microbreweri2, [m=1:M], sum(y[b,m] for b=1:B) <= StoreCap)

# At most one beer type per month
@constraint(microbreweri2, [m=1:M], sum(q[b,m] for b=1:B) <= 1)

optimize!(microbreweri2)
println("Objective: ", objective_value(microbreweri2)) # 192.5
```

Optimal value: €192.5

How this differs from the original Micro Brewery (§4.2)

The original (§4.2) is a pure LP — one beer type, brew as much as you like up to 120L. Micro Brewery 2 adds two complications: three competing beer types *and* the constraint that only one can be produced each month. The second constraint turns it into a MIP. This is the textbook Lot Sizing / Capacitated Production Planning problem.

Likely follow-up questions

1. **“Why is this a MIP and not an LP?”** The constraint “only one beer type per month” requires a binary variable. Without it, you could fractionally allocate capacity across types in the same month, which isn’t physically meaningful.
2. **“What is the role of $q_{\{b,m\}}$? Could you model this without it?”** $q_{\{b,m\}}$ acts as the selector binary. Without it, you can’t enforce “at most one type” linearly — there’s no way to say “ $x_{\{b,m\}} > 0$ for at most one b ” without introducing a binary.
3. **“Why 1 and not $= 1$ in constraint (iv)?”** Because the brewer might choose to brew nothing in a month if initial stocks are sufficient. Allowing $q_{\text{sum}} = 0$ is needed for feasibility in those months.
4. **“What if the storage cost differed by beer type?”** Change $\text{Cost} \cdot y$ to $\sum_b \text{Cost}_b \cdot \sum_m y_{\{b,m\}}$.
5. **“What if you could brew twice per month but still only one type per brewing?”** This problem becomes ambiguous (does “per brewing” mean anything?). If you mean at most 240L of one type, just change Capacity to 240. If you mean two separate brewing runs of potentially different types, you’d need two sets of y variables per month.
6. **“What would the LP relaxation look like, and is it tight?”** The LP relaxation lets $y_{\{b,m\}} \in [0,1]$. Since the optimal integer solution has $y_{\{b,m\}} \in \{0,1\}$ anyway (the LP is well-structured), the LP relaxation bound may be quite tight or even equal to the MIP optimum. This is often the case for lot sizing.
7. **“What happens if initial storage is 0 for all types?”** The model may become infeasible if demand in month 1 exceeds what can be produced (e.g. if total Jan demand > 120 L). You’d need to check: $\sum_b \text{Demand}_{\{b,1\}} \leq 120$. In this dataset $\sum_b \text{Jan demand} = 35+15+5 = 55 \leq 120$, so it’s fine.

§6.9 Workplan for Teaching Assistants — Assignments 9.1, 9.2

Classical OR problem: Manpower / shift scheduling.

Story (one sentence)

Four TAs must collectively cover known hourly demand across 9 days $\times$ 8 time slots, each working exactly 52 hours total, minimising their total inconvenience — with the added requirement (Assignment 9.2) that each TA’s daily schedule forms a single contiguous block of at least 2 hours.

Sets

- **ta** **TAs** = {AL, FR, JE, MI} ($T = 4$)
- **p** **Periods** = {1,...,8} (representing 9–10, 10–11, ..., 16–17)
- **d** **Days** = {1,...,9}

Parameters

- **Demand**_{p,d}: number of TAs required in slot p on day d
 - **Inconvenience**_{ta,p,d}: inconvenience score for TA ta working slot p on day d
-

Assignment 9.1 — Basic version

Decision variables

- **x**_{ta,p,d} ∈ {0,1}: 1 if TA ta works slot p on day d

Objective

$$\min \sum_{ta} \sum_p \sum_d Inconvenience_{ta,p,d} \cdot x_{ta,p,d}$$

Constraints

- (i) Cover demand in every slot:

$$\sum_{ta} x_{ta,p,d} \geq Demand_{p,d} \forall p, d$$

- (ii) Each TA works exactly 52 hours:

$$\sum_p \sum_d x_{ta,p,d} = 52 \forall ta$$

Julia/JuMP code

```
@variable(m, x[ta=1:T, p=1:P, d=1:D], Bin)
@objective(m, Min, sum(Inconvenience[ta,p,d]*x[ta,p,d] for ta=1:T, p=1:P, d=1:D))
@constraint(m, [p=1:P, d=1:D], sum(x[ta,p,d] for ta=1:T) >= Demand[p,d])
@constraint(m, [ta=1:T], sum(x[ta,p,d] for p=1:P, d=1:D) == 52)
```

Optimal value: 94.98

Assignment 9.2 — Add consecutive-block constraint

Each TA may work at most one contiguous block of hours per day, and if they work at all that day, they must work at least 2 hours.

New decision variable

- $y_{\{ta,p,d\}} \in \{0,1\}$: 1 if TA ta starts working in slot p on day d

New constraints (add to the Assignment 9.1 model)

- (iii) At most one start per TA per day:

$$\sum_p y_{ta,p,d} \leq 1 \forall ta, d$$

- (iv) Can only be working if already working or just started (sets $x_{\{ta,0,d\}} := 0$):

$$x_{ta,p,d} \leq (p > 1 ? x_{ta,p-1,d} : 0) + y_{ta,p,d} \forall ta, p, d$$

- (v) If working at all that day, work at least 2 slots:

$$\sum_p x_{ta,p,d} \geq 2 \cdot \sum_p y_{ta,p,d} \forall ta, d$$

Julia/JuMP code for new constraints

```
@variable(m, y[ta=1:T, p=1:P, d=1:D], Bin)
@constraint(m, [ta=1:T, d=1:D], sum(y[ta,p,d] for p=1:P) <= 1)
@constraint(m, [ta=1:T, p=1:P, d=1:D],
    x[ta,p,d] <= (p > 1 ? x[ta,p-1,d] : 0) + y[ta,p,d])
@constraint(m, [ta=1:T, d=1:D],
    sum(x[ta,p,d] for p=1:P) >= 2*sum(y[ta,p,d] for p=1:P))
```

Optimal value: 121.97 (worse than 9.1, as expected — fewer schedule arrangements available)

Likely follow-up questions

1. “Why does the inconvenience increase with the consecutive-block constraint?” Adding constraints to a minimisation problem can only weakly increase the optimum. The TAs lose the flexibility to work split shifts where convenient.
2. “What does $y_{\{ta,p,d\}}$ represent and why do you need it?” It marks the start of a TA’s working block. Without it, you can’t express “contiguous” linearly — you need an anchor point.
3. “Why 1 for starts, not = 1?” A TA might work zero hours on a day (no start, no work). Forcing = 1 would require them to work every day.
4. “Why == 52 in constraint (ii) and not 52?” Each TA is contracted and paid for exactly 52 hours. Using $\geq$ would let the solver minimise hours, possibly leaving demand uncovered.

5. “What if you wanted to minimise the maximum inconvenience across TAs instead of total?” Introduce z , add $\sum_{p,d} \text{Inconvenience}_{\{ta,p,d\}} \cdot x_{\{ta,p,d\}} \leq z$, and $\min z$ (minimax trick).

§6.12 Tennis — Assignments 12.1, 12.2, 12.3, 12.4

Classical OR problem: Chinese Postman Problem — find the shortest closed walk in a graph that traverses every required edge at least once.

Story (one sentence)

A sweeper must clean all lines on one half of a clay tennis court (12 intersection points, 11 required line segments) by walking every line at least once, minimising total walking distance.

Court layout

12 points with (x,y) coordinates (feet):

Label	Index	x	y
A0	1	0	0
B0	2	4.5	0
D0	3	31.5	0
E0	4	36	0
B1	5	4.5	18
C1	6	18	18
D1	7	31.5	18
A2	8	0	39
B2	9	4.5	39
C2	10	18	39
D2	11	31.5	39
E2	12	36	39

Required lines (oriented — defined one direction to avoid duplicates):

1→2, 1→8, 2→5, 2→3, 3→4, 3→7, 4→12, 5→6, 5→9, 6→7, 6→10, 7→11

Assignment 12.1 — Analytical (no model needed)

If only walking *on* the lines (no shortcuts), the optimal tour is **390 feet**. This is derived by inspection of the graph, not by MIP.

Assignment 12.2 — Walk between lines, return to start

Sets

- **p** **Points** = $\{1, \dots, 12\}$

Parameters

- **Distance_{p1,p2}**: Euclidean distance between points p1 and p2
- **Lines_{p1,p2}** $\{0,1\}$: 1 if there is a required line from p1 to p2

Decision variables

- **x_{p1,p2}** $\{0,1\}$: 1 if the sweeper walks from p1 to p2

Objective

$$\min \sum_{p1,p2} \text{Distance}_{p1,p2} \cdot x_{p1,p2}$$

Constraints

- (i) Flow conservation (valid walk — what comes in equals what goes out):

$$\sum_{p2} x_{p2,p1} = \sum_{p2} x_{p1,p2} \forall p1$$

- (ii) Every required line must be cleaned:

$$x_{p1,p2} + x_{p2,p1} \geq \text{Lines}_{p1,p2} \forall p1, p2$$

Optimal value: 306 feet

Assignment 12.3 — Allow start end

The sweeper doesn't need to return to their start point.

New decision variables

- **s_p** $\{0,1\}$: 1 if the sweeper starts at point p
- **e_p** $\{0,1\}$: 1 if the sweeper ends at point p

Modified flow conservation (replaces constraint (i)):

$$\sum_{p2} x_{p2,p1} + s_{p1} = \sum_{p2} x_{p1,p2} + e_{p1} \forall p1$$

New constraints

$$\sum_p s_p = 1(\text{exactly one start}) \quad \sum_p e_p = 1(\text{exactly one end})$$

Optimal value: 280.5 feet

Assignment 12.4 — Start and end must be on outside lines only

Interior nodes (B1, C1, D1, B2, C2, D2 = indices 5,6,7,9,10,11) are disallowed as start/end:

```
for p in [5,6,7,9,10,11]
    fix(s[p], 0; force=true)
    fix(e[p], 0; force=true)
end
```

Optimal value: 287.47 feet (slightly worse than 12.3, as expected — less flexibility)

Julia/JuMP code (Assignment 12.3, the most general version)

```
using JuMP, HiGHS

x_pos = [0, 4.5, 31.5, 36, 4.5, 18, 31.5, 0, 4.5, 18, 31.5, 36]
y_pos = [0, 0, 0, 0, 18, 18, 18, 39, 39, 39, 39, 39]
P = 12
Distance = [sqrt((x_pos[i]-x_pos[j])^2+(y_pos[i]-y_pos[j])^2) for i=1:P, j=1:P]

Lines = zeros{Int, P, P}
for (a,b) in [(1,2),(1,8),(2,5),(2,3),(3,4),(3,7),(4,12),(5,6),(5,9),(6,7),(6,10),(7,11)]
    Lines[a,b] = 1
end

m = Model(HiGHS.Optimizer); set_silent(m)
@variable(m, x[1:P,1:P], Bin)
@variable(m, s[1:P], Bin)
@variable(m, e[1:P], Bin)

@objective(m, Min, sum(Distance[p,q]*x[p,q] for p=1:P, q=1:P))

@constraint(m, [p=1:P],
    sum(x[q,p] for q=1:P) + s[p] == sum(x[p,q] for q=1:P) + e[p])
@constraint(m, sum(s[p] for p=1:P) == 1)
@constraint(m, sum(e[p] for p=1:P) == 1)
```

```

@constraint(m, [p=1:P, q=1:P], x[p,q] + x[q,p] >= Lines[p,q])

optimize!(m)
println("Distance: ", objective_value(m))

```

Likely follow-up questions

1. **“Why is this called the Chinese Postman Problem?”** Formulated by Mei-Ko Kwan: a postman walks every street at least once, returning home, minimising distance. The tennis sweeper is the same structure.
2. **“Why is the Lines matrix oriented (only one direction)?”** To avoid generating the same constraint twice. $x_{\{p1,p2\}} + x_{\{p2,p1\}}$ Lines_{\{p1,p2\}} covers both traversal directions with one constraint per pair.
3. **“What does flow conservation enforce?”** That the route is a valid continuous walk. Every time you enter a node, you must also leave it (except at the start and end under Assignment 12.3).
4. **“Why are $x_{\{p,p\}}$ allowed to exist in the variable declaration?”** They’re harmless — walking from a point to itself has zero distance and is never used in an optimal solution. You could exclude them with $[p=1:P, q=1:P; p!=q]$ for cleanliness.
5. **“Could you forbid traversing a specific line?”** $\text{fix}(x[p1,p2], 0)$ and $\text{fix}(x[p2,p1], 0)$, but also remove it from Lines (otherwise constraint (ii) becomes infeasible).
6. **“Why does the optimal distance increase from 12.3 to 12.4?”** Restricting feasible starts/ends reduces the solution space. Fewer options → the best option within the restriction is no better than before.

§6.14 Tariff Rates — Assignments 14.1, 14.2

Classical OR problem: Unit Commitment Problem — decide which generators to run in each time slot to meet electricity demand at minimum cost, accounting for startup costs and reserve requirements.

Story (one sentence)

Schedule 27 generators of 3 types across 5 daily time slots to meet electricity demand at minimum operating and startup cost, while always maintaining 15% reserve capacity — and the plan repeats every day, so it must be cyclic.

Data

Time slots and demand:

Slot	Hours	Duration	Demand (MW)
1	00:00–06:00	6h	15,000
2	06:00–09:00	3h	30,000
3	09:00–15:00	6h	25,000
4	15:00–18:00	3h	40,000
5	18:00–24:00	6h	27,000

Generator specifications:

Type	Count	Min (MW)	Max (MW)	Cost/h at min (€)	Extra cost/(h · MW) (€)	Startup cost (€)
1	12	850	2,000	1,000	2	2,000
2	10	1,250	1,750	2,600	1.3	1,000
3	5	1,500	4,000	3,000	3	500

Key: Generators within the same type are **indistinguishable** — you only need to track *how many* of each type are running, not *which ones*.

Reserve requirement: At all times, total *available* generating capacity must be $1.15 \times$ current demand. (Available capacity = max output of running generators, not actual output.)

Sets

- **p** **PlantTypes** = {1, 2, 3}
- **h** **HourPeriods** = {1, ..., 5}

Parameters

- **PDEM_h**: MW demand in period h
- **NHP_h**: number of hours in period h (6, 3, 6, 3, 6)
- **NG_p**: number of generators of type p (12, 10, 5)
- **MINL_p**, **MAXL_p**: min and max MW per generator of type p
- **MINC_p**: €/hour cost to run one generator at minimum level
- **PC_p**: extra €/(hour · MW) above minimum
- **STARTC_p**: one-time startup cost per generator of type p that starts

Decision variables

- **nRun_{p,h}** $\mathbb{Z}, 0$ **nRun** **NG_p**: number of generators of type p Running in period h
- **prod_{p,h}** 0 : total MW **produced** by all type-p generators in period h

- **nStart_{p,h}** 0: number of type-p generators that **Start** up in period h

These names are not the teacher's (x/y/s) but are semantically unambiguous. Any examiner will accept them — what matters is that you define each variable clearly in words before using it.

Objective

Minimise total operating + startup cost:

$$\min \sum_h \sum_p MINC_p \cdot NHP_h \cdot nRun_{p,h} (fixedruncost) + \sum_h \sum_p PC_p \cdot NHP_h \cdot (prod_{p,h} - MINL_p \cdot nRun_{p,h}) (variable)$$

Constraints

- (i) **Minimum production** — a running generator must produce at least its minimum:

$$prod_{p,h} \geq MINL_p \cdot nRun_{p,h} \forall p, h$$

- (ii) **Maximum production** — a running generator cannot exceed its maximum:

$$prod_{p,h} \leq MAXL_p \cdot nRun_{p,h} \forall p, h$$

- (iii) **Meet demand** — total actual production covers demand:

$$\sum_p prod_{p,h} \geq PDEM_h \forall h$$

- (iv) **Startup counting** — nStart_{p,h} counts generators that turn ON in period h. The plan is **cyclic** (period 0 = period H):

$$nStart_{p,h} \geq nRun_{p,h} - nRun_{p,h-1} \forall p, h (with nRun_{p,0} := nRun_{p,H})$$

- (v) **Reserve capacity** — enough generators running to *potentially* meet 115% of demand:

$$\sum_p MAXL_p \cdot nRun_{p,h} \geq 1.15 \cdot PDEM_h \forall h$$

Julia/JuMP code

`using JuMP, HiGHS`

`H = 5; P = 3`

`NG = [12, 10, 5]`

`NHP = [6, 3, 6, 3, 6]`

`PDEM = [15000, 30000, 25000, 40000, 27000]`


```

MINL    = [850, 1250, 1500]
MAXL    = [2000, 1750, 4000]
MINC    = [1000, 2600, 3000]
PC      = [2, 1.3, 3]
STARTC  = [2000, 1000, 500]

tarifrates = Model(HiGHS.Optimizer); set_silent(tarifrates)

@variable(tarifrates, 0 <= nRun[p=1:P, h=1:H] <= NG[p], Int) # generators running
@variable(tarifrates, prod[1:P, 1:H] >= 0) # total production (MW)
@variable(tarifrates, nStart[1:P, 1:H] >= 0) # generators started up

@objective(tarifrates, Min,
    sum(MINC[p] * NHP[h] * nRun[p,h] for p=1:P, h=1:H) +
    sum(PC[p] * NHP[h] * (prod[p,h] - MINL[p]*nRun[p,h]) for p=1:P, h=1:H) +
    sum(STARTC[p] * nStart[p,h] for p=1:P, h=1:H))

# Min and max production per running generator
@constraint(tarifrates, [p=1:P, h=1:H], prod[p,h] >= MINL[p]*nRun[p,h])
@constraint(tarifrates, [p=1:P, h=1:H], prod[p,h] <= MAXL[p]*nRun[p,h])

# Meet demand
@constraint(tarifrates, [h=1:H], sum(prod[p,h] for p=1:P) >= PDEM[h])

# Startup counting - cyclic: period 0 = period H
@constraint(tarifrates, [p=1:P, h=1:H],
    nStart[p,h] >= nRun[p,h] - (h > 1 ? nRun[p,h-1] : nRun[p,H]))

# 15% reserve capacity (Assignment 14.1 only - remove for 14.2)
@constraint(tarifrates, [h=1:H],
    sum(MAXL[p]*nRun[p,h] for p=1:P) >= 1.15*PDEM[h])

optimize!(tarifrates)
println("Total cost: ", objective_value(tarifrates)) # 988,540

```

Optimal values

- **Assignment 14.1** (with 15% reserve): **€988,540**
- **Assignment 14.2** (savings from removing 15% reserve): **€750** (new optimum: €987,790)

Common pitfall

If you get **€1,015,150** instead of €988,540, you forgot the cyclic constraint. Without $x_{\{p,0\}} := x_{\{p,H\}}$, period 1 assumes zero generators were running “yesterday”, incurring full startup costs for all generators in slot 1.

Likely follow-up questions

1. **“Why use `prod = total production`, not `extra-above-minimum`?”**
Both formulations work. Using total production keeps the demand constraint simple (`prod demand`) and separates the min/max bounds cleanly into two constraints. The cost above minimum then appears in the objective as $PC \cdot (prod - MINL \cdot nRun)$.
2. **“Why are generators of the same type interchangeable?”** Identical min/max levels and costs — we only need to know *how many* are running (`nRun`), not *which ones*. This makes `nRun` an integer count rather than a binary per individual machine.
3. **“What does the reserve constraint enforce, and why does removing it save €750?”** It forces enough generators running so we *could* serve 115% of demand — even if `prod` is less. The saving (€750) is the cost of this insurance.
4. **“What is the cyclic constraint and why does forgetting it inflate the cost?”** Without it, the model treats “before period 1” as `nRun = 0`, so period 1 incurs full startup costs for all running generators. The cyclic version correctly reads `nRun` from period H (previous day’s last period) instead.
5. **“Why is `nStart` declared continuous, not integer?”** Because $nStart[p,h] = nRun[p,h] - nRun[p,h-1]$ and `nStart 0` together force `nStart` to integer values at optimum anyway (since `nRun` is integer). Continuous is faster for the solver.
6. **“Assignment 14.2: how do you find the saving?”** Comment out constraint (v) and re-solve. The difference between the two objective values is the cost of the reserve requirement.

§6.17 The Wedding Planner — Assignments 17.1, 17.2, 17.3, 17.4, 17.5

Classical OR problem: Multi-objective assignment with side constraints. Built incrementally across 5 sub-assignments — each adds a new goal.

Story (one sentence)

Assign 20 wedding guests to 3 tables (capacity 9 each), keeping couples together, while maximising shared interests at each table, balancing genders, and ensuring each guest knows enough others at their table.

Sets and Parameters (used across all sub-assignments)

- `g` Guests = {1,...,20}

- t Tables = $\{1,2,3\}$, TableCap = 9
 - Couple_ $\{g1,g2\}$: 1 if $g1$ and $g2$ are a couple
 - SharedInterests_ $\{g1,g2\}$: integer (from Assignment 17.2 onward)
 - Male_ g , Female_ g : gender flags (from Assignment 17.3 onward)
 - Know_ $\{g1,g2\}$: 1 if $g1$ and $g2$ know each other (from Assignment 17.4 onward)
-

Assignment 17.1 — Feasibility only

Find *any* valid seating. No real objective (use `min 42` as a placeholder).

Variables

- $x_{\{g,t\}} \in \{0,1\}$: 1 if guest g sits at table t

Constraints

- Every guest at exactly one table: $\sum_t x_{\{g,t\}} = 1 \quad g$
 - Table capacity: $\sum_g x_{\{g,t\}} \leq 9 \quad t$
 - Couples together: $x_{\{g1,t\}} = x_{\{g2,t\}} \quad t, (g1,g2) \text{ with Couple}_{\{g1,g2\}}=1$
-

Assignment 17.2 — Maximise shared interests

New variable

- $y_{\{g1,g2,t\}} \in [0,1]$ (continuous, defined only for $g1 < g2$): = 1 if both $g1$ and $g2$ sit at table t

New objective

$$\max \sum_t \sum_{g1 < g2} \text{SharedInterests}_{g1,g2} \cdot y_{g1,g2,t}$$

New constraints (linearisation of binary product)

$$y_{g1,g2,t} \leq x_{g1,t} \quad \forall t, g1 < g2 \quad y_{g1,g2,t} \leq x_{g2,t} \quad \forall t, g1 < g2$$

(The third linearisation constraint $y_{\{g1\}} + x_{\{g2\}} - 1$ is omitted because the maximisation objective already pulls y up whenever allowed.)

Optimal value: 67

Assignment 17.3 — Balance genders

Target: at most 2 more men than women per table (and vice versa). Use slack variables for soft enforcement (so the model remains feasible even if perfect balance is impossible).

New variables

- **m_t** 0: excess males beyond +2 at table t
- **f_t** 0: excess females beyond +2 at table t

New constraints

$$\sum_g (Male_g \cdot x_{g,t} - Female_g \cdot x_{g,t}) \leq 2 + m_t \forall t \quad \sum_g (Female_g \cdot x_{g,t} - Male_g \cdot x_{g,t}) \leq 2 + f_t \forall t$$

New objective (when running this stage alone)

$$\min \sum_t (m_t + f_t)$$

Optimal value (this objective alone): 0 (perfect gender balance is achievable)

Assignment 17.4 — Each guest knows 3 others at their table

New variable

- **k_g** 0: shortfall in “known guests at same table” for guest g below 3

New constraint

$$k_g - 3 \cdot x_{g,t} + \sum_{g1} Know_{g,g1} \cdot (y_{g,g1,t} + y_{g1,g,t}) \geq 0 \forall g, t$$

At the table where g sits ($x_{\{g,t\}}=1$), k_g must cover any shortfall between the 3 required and the number of known guests actually at that table. The $y_{\{g,g1,t\}} + y_{\{g1,g,t\}}$ term handles the fact that y is only defined for $g1 < g2$.

New objective (this stage alone)

$$\min \sum_g k_g$$

Optimal value (this stage alone): 2

Assignment 17.5 — Combined multi-objective

All three goals combined into one weighted-sum objective:

$$\min - \sum_t \sum_{g1 < g2} SharedInterests_{g1,g2} \cdot y_{g1,g2,t} + \sum_t (m_t + f_t) + \sum_g k_g$$

(Shared interests is negated because we want to *maximise* it, but everything is in a *min* objective.)

Optimal value: −60

Julia/JuMP code (full Assignment 17.5 model)

```
using JuMP, HiGHS
include("WeddingData20.jl") # G, T, TableCap, Couple, SharedInterests, Male, Female, Know

m = Model(HiGHS.Optimizer); set_silent(m)

@variable(m, x[1:G, 1:T], Bin)
@variable(m, 0 <= y[g1=1:G, g2=1:G, t=1:T] <= (g1 < g2 ? 1 : 0))
@variable(m, mm[1:T] >= 0) # excess males
@variable(m, ff[1:T] >= 0) # excess females
@variable(m, k[1:G] >= 0) # knowing shortfall

@objective(m, Min,
    -sum(SharedInterests[g1,g2]*y[g1,g2,t] for g1=1:G,g2=1:G,t=1:T if g1<g2)
    + sum(mm[t]+ff[t] for t=1:T)
    + sum(k[g] for g=1:G))

@constraint(m, [g=1:G], sum(x[g,t] for t=1:T) == 1)
@constraint(m, [t=1:T], sum(x[g,t] for g=1:G) <= TableCap)
@constraint(m, [g1=1:G, g2=1:G, t=1:T; Couple[g1,g2]==1], x[g1,t] == x[g2,t])

@constraint(m, [t=1:T],
    sum(Male[g]*x[g,t]-Female[g]*x[g,t] for g=1:G) <= mm[t]+2)
@constraint(m, [t=1:T],
    sum(Female[g]*x[g,t]-Male[g]*x[g,t] for g=1:G) <= ff[t]+2)

@constraint(m, [g1=1:G,g2=1:G,t=1:T; g1<g2], y[g1,g2,t] <= x[g1,t])
@constraint(m, [g1=1:G,g2=1:G,t=1:T; g1<g2], y[g1,g2,t] <= x[g2,t])

@constraint(m, [g=1:G,t=1:T],
    k[g] - 3*x[g,t] + sum(Know[g,g1]*(y[g,g1,t]+y[g1,g,t]) for g1=1:G) >= 0)
```

```
optimize!(m)
println("Objective: ", objective_value(m)) # Should be -60
```

Likely follow-up questions

1. **“Why is y declared continuous, not binary?”** Because the linearisation constraints bound y between 0 and 1, and the maximisation objective pulls it to 1 whenever allowed. The LP relaxation is automatically tight here.
2. **“Why only define y for $g1 < g2$?”** To avoid counting each pair twice. $\text{SharedInterests}_{\{g1,g2\}}$ would otherwise contribute to the objective via $y_{\{g1,g2,t\}}$ and $y_{\{g2,g1,t\}}$.
3. **“Why omit the third linearisation constraint?”** The objective is maximising y (negated in min), so y will always be pushed to its upper bound. The third constraint would only add redundant information.
4. **“What do the slack variables m_t and f_t do?”** They soften the gender balance constraint. Without them, any instance where perfect balance is impossible would be infeasible. With them, imbalance is allowed but penalised.
5. **“What does a combined objective value of -60 mean physically?”** It’s $\text{shared_interests} - \text{gender_slack} - \text{knowing_slack} = 64 - 2 - 2 = 60$, so the objective is -60 . The negative sign is because shared interests enter the objective negated.
6. **“How would you weight shared interests $3\times$ more than gender balance?”** Multiply: $\min -3 \cdot (\text{SI} \cdot y) + 1 \cdot (m+f) + 1 \cdot (k)$.

7. Whiteboard presentation strategy

Using your 30 minutes of prep

1. **0–3 min:** Read the problem statement **twice**. Underline the objective, the constraints, any specific numbers.
2. **3–7 min:** Identify the classical OR problem. Write the 5 headings.
3. **7–18 min:** Fill in each section on paper. Write constraints in plain English first, then in math.
4. **18–25 min:** Sanity check. For each constraint, does the math say what the English says?
5. **25–30 min:** Plan the whiteboard layout. Where will the table go? Where the constraints?

At the whiteboard

- **Write the 5 headings first**, then fill in. Never dive straight into constraints.
- **Talk while you write.** “I’ll define $x_{\{g,t\}}$ as 1 if guest g sits at table t ...”
- **Name the classical problem early:** “This is a variant of the Unit Commitment Problem...”
- **Explain each constraint in plain English** as you write it.
- **If you forget a detail**, reason through it aloud: “I don’t remember the exact number, but it must be at least the maximum demand across all slots, so...”

Numbers to memorise

Assignment	Optimal value
§6.5 Assignment 5.1 — Micro Brewery 2	€192.5
§6.9 Assignment 9.1 — TA Workplan basic	94.98
§6.9 Assignment 9.2 — TA Workplan w/ blocks	121.97
§6.12 Assignment 12.1 — Tennis analytical	390 ft
§6.12 Assignment 12.2 — Tennis start=end	306 ft
§6.12 Assignment 12.3 — Tennis start end	280.5 ft
§6.12 Assignment 12.4 — Tennis outside only	287.47 ft
§6.14 Assignment 14.1 — Tariff Rates w/ reserve	€988,540
§6.14 Assignment 14.2 — Tariff Rates saving	€750
§6.17 Assignment 17.2 — Wedding interests	67
§6.17 Assignment 17.3 — Wedding gender	0
§6.17 Assignment 17.4 — Wedding knowing	2
§6.17 Assignment 17.5 — Wedding combined	−60

8. Self-test — try these without your notes

Once you’ve studied, sit with blank paper and produce the model for each in under 12 minutes.

1. **Micro Brewery 2 variant:** Now you can brew two types per month, but each brewing run is capped at 80L, and the total brewed per month is still 120L. Adapt the model.
2. **TA Workplan variant:** TAs can now skip up to 2 days of work (not work at all that day), but must still hit 52 hours total. How does the model change?
3. **Tennis variant:** Two sweepers must together clean all lines. Each can start and end anywhere. Minimise total walking distance of both. (Hint:

introduce a second set of x variables.)

4. **Tariff Rates variant:** A new generator type 4 costs €4,000/h at minimum and has no startup cost. How do you add it? What happens to the optimal value?
 5. **Wedding Planner variant:** The bride demands that no two specific guests (say, $g_1=3$ and $g_2=7$, ex-partners) sit at the same table. Add this constraint to the Assignment 17.5 model.
-

9. Final reminders

- Upload the **SIP production planning** problem and data file so I can help you prepare Part 1 of the oral.
- The **5 headings** (Sets / Parameters / Variables / Objective / Constraints) are your whiteboard anchor. Write them first, every time.
- Notes are allowed in prep — bring a clean summary of each model. But practise presenting *without* reading them.
- For the oral: **structure beats speed**. A calm, step-by-step explanation of a correct model beats a rushed presentation of a flawed one.
- If you blank: stop, take a breath, write the 5 headings, start filling in the Sets. The rest follows.

Good luck.